

Analysis: Havannah

This problem probably looked a little scary to many competitors at first. (I, for one, remember some nasty problems involving hexagons in other contests). However, this one wasn't as bad as it looked.

Let's first see what kind of cells there are and how to distinguish between them:

- **Inner Cells:** Those were the cells that are not part of the border of the board. They had 6 neighboring cells.
- **Corner Cells:** Those were the cells where two edges of the board overlap. They had 3 neighboring cells.
- **Edge Cells:** Those were the cells that were neither inner cells, nor corner cells. They had 4 neighboring cells.

Note that it was a bit counter-intuitive that the corner cells were *not* edge cells (as they, in fact, lie on the edges), however that was clearly defined in the statement: "Corners do not count as edges".

We can enumerate the edges and the corners of the board. Each edge will have a distinct integer index between 0 and 5, inclusive, and each corner will also have a distinct index between 0 and 5, inclusive. This is okay, but for simplicity later we will give the indices from 0 to 5 to the corners and add 6 to the indices of the edges, to obtain 6 to 11 for the edges. If a cell neither is a corner, nor belongs to any of the edges, we can mark it with -1. Thus, we create a function `int checkType(row, col)` that checks if the cell at (row, col) is a corner (and which one if it is), part of an edge (and which edge, if it is) or neither.

- Corners are the cells: $\{(1, 1), (1, S), (S, S * 2 - 1), (S * 2 - 1, S * 2 - 1), (S * 2 - 1, S), (S, 1)\}$
- Edges are the cells: $\{(1, X), (X, 1), (X, S * 2 - 1), (S * 2 - 1, X)\}$ where X is any integer, together with all cells for which $|row - col| == S - 1$.

Don't forget that corners are not considered edges, so exclude them when checking for edges.

Now let's examine in more detail the winning figures:

- **Bridge:** This was probably the simplest of the three: just a set of stones that connects any two corner cells. The minimal number of stones to achieve it was 2 on a board with $S = 2$ (which was the minimal possible board).
- **Fork:** This was a bit more complicated. We need a set of stones, that connects 3 distinct edges (i.e. cells from three distinct edges). The minimal number of stones to achieve it was 5 on a board with $S = 3$ (note that the board with $S = 2$ does not have any edge cells).
- **Ring:** This was the most complicated figure. We need a set of stones that encircles an empty cell. As it might have been a bit unclear what "encircles" in this case means, the problem statement contained a detailed explanation and an example. The minimal number of stones to achieve it was 6 on a board with $S = 3$. Note that a ring on a board with $S = 2$ is impossible, as it inevitably would lead to a bridge first.

As many of the competitors might have realized, the Bridge and Fork cases are relatively simple to detect. One way to solve them would be to use binary search over the move in which they are created and check if a Bridge and/or a Fork exists. That was okay, however it didn't work for Rings. Some competitors used this solution for Forks and Bridges, and another one for Rings. However, this required some extra code, which is rarely a good idea in a speed contest.

Some competitors probably realized that even an $O(M^2)$ solution would pass. However, we will describe an $O(M)$ solution, which was very fast and not much harder to implement.

First, it's a good idea to store the board in a both memory and time efficient way. The thing to notice was that when the board is really big, then it is also really sparse (only 10000 stones in a board with over 9000000 cells). So the way to go was to use any kind of set data structure our programming language of choice provided. In C++, a hashset is fastest, but tree sets are also fine. Putting a stone is a constant operation if we are using a hashset and logarithmic one if we're using balanced tree set (for example C++ STL set).

Now we start putting stones on the board in the order they are given. We need a fast way to check if putting a stone created some of the winning figures. The fork and the bridge needed some cells to be connected (i.e. to belong to the same connected component). In fact, the ring also requires the same thing. As many competitors in this round probably know, these are usually handled using the [union find](#) algorithm. As a reminder union-find supports the following basic operations:

- Create a new node, belonging to a group of size 1.
- Merge two new groups into one.
- For a given node, find the group it is.

Implemented correctly, all of these operations are blazingly fast.

We will use union-find here with a minor addition. After putting a stone, we see if some of the neighboring cells contains a stone already. If none do, we add the new stone to its own connected component. If there are stones in adjacent squares, we merge them all to a single connected component that also contains the new stone. Additionally to the straight-forward union-find algorithm, we will store what type of cells each component contains. Since the number of different types of cells we are interested in is relatively small (only 12 types) we can use a mask with 12 bits (corresponding to the indices from 0 to 11 we mentioned earlier). We do this in order to have an easy way to see if a component contains corner or edge cells. When merging the component A to component B, we change the mask of B to contain the bits of the mask of A as well (since now they are the same component). This can be done really easily with the bitwise operation OR. If, after merging, a component ends up with 2 set bits in the first 6, or 3 set bits in the second 6 positions, then we've just created a bridge or a fork, respectively.

Everything's fine with that, but we still haven't answered the question how do we handle rings. Well, having the stones and their respective components makes it easy to check for them. In order for a ring to be created, we must have just added a stone that connects a component to itself. But if it does, it still does not necessarily create a ring. What we can do is check all neighbors of the cell where we are putting the stone, and search for two cells with stones with the same component.

We can represent the neighbors of the given cell as the following:

```
# 1 2
6 * 3
5 4 #
```

Going clockwise twice (or counter-clockwise, if you prefer, it doesn't matter) and looking at the neighbors there should be one of the following sequences (as a subsequence of the created sequence of 12):

1. {C, X1, C, Y1, Y2, Y3}
2. {C, X1, X2, C, Y1, Y2}
3. {C, X1, X2, X3, C, Y1}

where the cells C belong to the same component, and each of the cells X and Y are either empty or belong to some component (not necessarily the same as C, and not necessarily the same as other Xs and Ys).

After this move a ring is formed if and only if:

- At least one of the Xs is an empty cell
- At least one of the Ys is an empty cell

(Note that if there is a $\{C, X1, C, Y1, Y2, Y3\}$ sub-sequence, then there will be a $\{C, X1, X2, X3, C, Y1\}$ one, however we've included them both for clarity).

Why is this true? Well, if none of the Xs or none of the Ys is an empty cell, then the two Cs were already connected "from this side" and adding the stone doesn't change it into a ring (obviously). If both some of the X cells and some of the Y cells contain an empty cell, then we've just encircled at least one of them! Imagine it this way - a circle has two sides - inside and outside. We know that we created a circle (since we are connecting a component to itself), but we don't know which side is the inner one and which side is the outer one. Thus, if both contain an empty cell, then we have an empty cell in the inner side for sure.

What is the time complexity of the given algorithm? Since OR is a constant operation, we have a constant number of bits to check when looking for a Fork or a Bridge, and checking for a ring involves also a constant number of operations (equal to the number of neighbors), the complexity is dominated by the speed of the union-find operations. Thus, the total complexity of the algorithm is $O(M * \text{RACK}(M))$, where $\text{RACK}()$ is the inverse of the Ackerman function. However, $\text{RACK}()$ is so slowly growing, that you can assume the algorithm will run in nearly $O(M)$ for the given constraints.

Remark: Another nice trick for dealing with rings is to start with all pieces played on the board and work backwards, always considering which empty squares are connected to each other. Removing a stone possibly connects two of these components, and so we can again use union-find to track the state quickly. This is conceptually simpler, but it is slower because we need to first find all the connected components after the pieces are played.

Analysis: Lost Password

This is certainly an intimidating problem. Even small strings, like the example from The Fellowship of the Ring, are difficult to work through. When you are trying to combine passwords of length at most 500 into a password string, there are so many possibilities to consider that optimization quickly becomes overwhelming.

The Small Input

Let's begin with the small input. In this case, we need to connect up pairs of letters. The key insight here is to imagine everything as a graph. If you have heard of [De Bruijn sequences](#) before, that is the perfect thing to build from. We will make a vertex for each letter (the 26 normal letters as well as all the 133t variations), and add an edge between every pair of letters. Each 2-letter word that we need in our password string corresponds to an edge on this graph. For example, "t0" corresponds to the edge 't' -> '0'. Let's call these *candidate edges*.

Next let's consider a password string. We can think of this as a path on the graph. We start at the first letter, and then continually move to the next letter in the string. For example, "abc0" corresponds to the path 'a' -> 'b' -> 'c' -> '0'. Therefore, the problem can be re-stated as follows: what is the length of the shortest path on this graph that includes all the candidate edges?

Here it is helpful to think back to another classic algorithm problem: [Eulerian paths](#). The problem could also be re-stated as: if we start with just the candidate edges, what is the minimum number of edges we need to add so that the resulting graph has an Eulerian path? Fortunately for us, the Eulerian path problem has been completely solved!

Fact: A directed graph has an Eulerian path if and only if (1) every vertex has in-degree equal to its out-degree except possibly for two vertices that are off by one, and (2) every vertex with positive degree is connected in the underlying undirected graph.

If you play with some examples, you will see that you will always be connected here, but let's come back to a formal proof when we talk about the large input. The fact that connectivity comes for free is really what makes this problem solvable, and so it is a good thing to think about!

The remaining condition says that all we need to do is add edges to balance in-degrees and out-degrees. We can add any edge we want at any time, so the method is simple: choose a vertex *u* with higher in-degree than out-degree and a vertex *v* with higher out-degree than in-degree, and then add an edge from *u* to *v*. Repeat until there are only two vertices left that are only slightly imbalanced, and we're done! After all that talk, we end up with a very simple greedy algorithm.

The Large Input

In fact, this solution already has most of the key ideas that go into solving the large as well, but we just need to take them further. There are three main challenges:

- When passwords are length greater than 2, how do we interpret them as edges on a graph?

- When it comes time to balancing degrees, some edges will be impossible to add. How do we adapt the greedy algorithm from before?
- The output password could be huge! Is it really possible to solve the problem in time less than the size of the output password?

The first challenge is the most important, and again, De Bruijn sequences provide a good model to build from. If we are trying to construct all passwords of length k , we will create a graph with vertices corresponding to all length- $(k-1)$ phrases. Each candidate password corresponds to the edge between its prefix phrase and its suffix phrase. Let's begin by making this nice and rigorous, although the intuition is exactly the same as in the small input.

A Minimum-Cost Flow Interpretation

Let us call any length- k substring of $\mathbf{S}$ (and any of its "133tspeak" equivalents) a "candidate", and any length- $(k-1)$ string composed of digits and lowercase letters a "phrase" (whether it is contained in a candidate or not). For each phrase s , define its weight $w(s)$ to be the number of candidates that end with s minus the number of candidates that begin with s . Note that the sum of $w(s)$ is 0. (The weight of a phrase will measure how far the corresponding vertex is from supporting an Eulerian path.)

Form a directed graph G with vertices corresponding to the phrases. For phrases a and b , add an edge from a to b if you can make a into b by adding a letter to the end of a and removing the first letter of a . Now we set up a flow problem: if a phrase s has positive weight, it is a source with capacity $|w(s)|$; otherwise it is a sink with capacity $|w(s)|$. All edges have infinite capacity and cost one.

Let ANSWER denote the shortest possible length of a password string for $\mathbf{S}$, let C denote the set of all candidates, and let FLOW_EDGES denote the minimum number of edges (i.e. minimum cost) in an "almost-maximum" flow. (Specifically, this is a flow that leaves at most 1 capacity unused in some source and at most one 1 capacity unused in some sink).

Lemma 1: $\text{ANSWER} = \text{FLOW_EDGES} + |C| + k - 1$.

Proof Part 1: $\text{ANSWER} \leq \text{FLOW_EDGES} + |C| + k - 1$

Let G' be the directed graph formed as follows:

- Begin with the directed multi-graph formed by the minimum-cost almost-maximum flow on G .
- For each candidate c , add an edge from the prefix phrase of c to the suffix phrase of c .

For a given phrase s , let's calculate its out-degree minus its in-degree in G' . After the first step, this value is exactly $w(s)$, except for two vertices that are off by 1. (This is due to the fact that our flow is only almost maximum.) After the second step, this value becomes exactly 0, again except for two vertices that are off by 1.

Therefore, we know G' satisfies the condition on in-degrees and out-degrees for it to have an Eulerian path. (See the "Fact" in the Small Input discussion.) We now show it also satisfies the connectivity condition. This is where we use the specific nature of the problem and the fact that $|\mathbf{S}| \geq 2k$. Actually, we suspect the last condition is unnecessary in the end, but the proof becomes much harder if this is not true!

Let's say a vertex s is a *core* vertex if it corresponds to a phrase in $\mathbf{S}$, or to a 133tspeak variation of such a phrase.

- If s is a core vertex, then s is adjacent in G' to its predecessor and successor phrases within $\mathbf{S}$. (Note that there may be multiple predecessors or successors if the new letter has a leet variation or if $\mathbf{S}$ appears multiple times.) Therefore, we can certainly follow the edges of G' to go from s to some phrase $a(s)$ that starts at the first letter of $\mathbf{S}$. We can then walk from there to a phrase $b(s)$ that ends at the last letter of $\mathbf{S}$. Since $a(s)$ and $b(s)$ are completely disjoint, we can choose $b(s)$ to be completely non-leet by always adding on non-leet successor letters. This means $b(s)$ does not depend on s , and hence we have demonstrated every core vertex is connected to a single vertex in G' .
- Now consider a non-core vertex t with positive degree in G' . This can only happen if t has positive degree in G , and therefore it must be connected via the flow to some core vertex s . Since we just showed all core vertices are connected, we conclude the non-core vertices are connected as well.

Therefore, the underlying undirected graph of G' is indeed connected, and hence G' has an Eulerian path consisting of some vertices $s_1, s_2, \dots, s_{\text{FLOW_EDGES} + |C| + 1}$. We can now construct a password string as follows:

- Begin the password string with the $k - 1$ letters in s_1 .
- For each subsequent vertex s_i , append to the password string the one letter at the end of s_i that is not in s_{i-1} .

This string has length exactly $k - 1 + \text{FLOW_EDGES} + |C|$, as required. Notice that after appending the letter for s_i , the final $k-1$ letters in the password string are always precisely equal to s_i . (This invariant can easily be proven by induction.) Now consider an arbitrary candidate c . Because of how we constructed G' , c has prefix s_{i-1} and suffix s_i for some i . But then after appending the letter for s_i , the last k letters precisely spell out c . Hence, every candidate is in this password string, and the inequality is proven.

Proof Part 2: $\text{ANSWER} \geq \text{FLOW_EDGES} + |C| + k - 1$

For the second part of the proof, we just need to reverse the previous argument. It is actually a little easier because we do not have to worry about connectivity.

Consider a password string P of length ANSWER . By definition, we know each candidate must appear somewhere in P . Therefore, for each candidate c , we can define $\text{pos}(c)$ to be the character in P where the first occurrence of c begins. Now let's order the candidates $c_1, c_2, \dots, c_{|C|}$ by $\text{pos}(c)$.

For each i , consider the substring of P starting with character $\text{pos}(c_i) + 1$ and ending with character $\text{pos}(c_{i+1}) + k - 2$. Note that this substring begins with the suffix phrase of c_i and ends with the prefix phrase of c_{i+1} .

By reversing the construction from the previous section, we can interpret this substring as a path in the graph G from the suffix phrase of c_i to the prefix phrase of c_{i+1} . This path has $\text{pos}(c_{i+1}) - \text{pos}(c_i) - 1$ edges in it. Now let's think about what happens when we combine all these paths. We get a flow with a source at every suffix phrase except $c_{|C|}$ and a sink at every prefix phrase except c_1 . When we add all these sources and sinks together, we get precisely an almost max-flow on G . Furthermore, this flow uses exactly $\sum_i [\text{pos}(c_{i+1}) - \text{pos}(c_i) - 1] = \text{pos}(c_{|C|}) - \text{pos}(c_1) - |C| + 1$ edges.

It follows that $\text{FLOW_EDGES} \leq \text{pos}(c_{|C|}) - \text{pos}(c_1) - |C| + 1$. Finally, we know $\text{pos}(c_{|C|}) \leq |P| - k = \text{ANSWER} - k$, and also $\text{pos}(c_1) \geq 0$. Therefore, $\text{ANSWER} \geq \text{FLOW_EDGES} + k + |C| - 1$, as required.

A Greedy Solution

At this point, we still have not solved the whole problem, but we have reduced it to something more tractable. Minimum-cost flow is a complicated problem but it is well known, and at least it can be solved in polynomial time. We can either try to optimize here, or we can use more clever idea to make our life much simpler.

Lemma 2: Fix the graph G and assign arbitrary source and sink capacities to arbitrary nodes. Then, a minimum-cost almost-max flow can be achieved by repeatedly taking the shortest path from an unused source to an unused sink, and pushing flow along there (without ever reverting any flow that's already been pushed).

Proof: Let F denote a minimum-cost almost-max flow on G , and suppose the shortest path in G between a source and a sink goes from source u to sink x . Furthermore, suppose that F contains a path from u to a different sink y , and a path from a different source v to x . We claim that we could replace these two paths with a path from u to x and with a path from v to y to achieve a flow with no more edges. (Since F is only an almost-max flow, it might also be that u and/or x is simply not used in F , but that case is a lot simpler.) Recall that every edge in G has infinite capacity, so the only challenge here is making sure the path lengths work out.

Given two phrases p and q , let's define $A(p, q)$ to be the longest string that is both a suffix of p and a prefix of q . Then the distance from p to q in G is precisely equal to $k - 1 - |A(p, q)|$. This means we can reformulate the claim as follows: given that $|A(u, x)| \geq \max(|A(u, y)|, |A(v, x)|)$, prove that $|A(u, x)| + |A(v, y)| \geq |A(u, y)| + |A(v, x)|$.

Now, notice that $A(u, x)$ and $A(u, y)$ are both suffixes of u , but $A(u, x)$ is at least as long. Therefore, $A(u, y)$ is a suffix of $A(u, x)$. Similarly, $A(v, x)$ is a prefix of $A(u, x)$. Let $t = |A(u, y)| + |A(v, x)| - |A(u, x)|$. If $t \leq 0$, the claim is trivial. Otherwise, there must be a length- t string z that is a suffix of $A(v, x)$ and a prefix of $A(u, y)$. Then z is also a suffix of v and a prefix of y . Therefore, $|A(v, y)| \geq |z| = t$, and the claim is proven.

We have shown that F can be modified to contain the path from u to x without increasing the number of edges. Now consider the rest of F . It defines a smaller flow, and we can repeat the same argument to show this residual flow can also be modified to contain the shortest path between a remaining source and a remaining sink, and that this modification will not increase the number of edges. Applying this argument repeatedly gives us the flow proposed in the lemma without ever increasing the number of edges, and so we have shown that this flow is indeed optimal.

The Implementation

Almost there! We have outlined a greedy algorithm, but what does it actually mean when it is put back into the context of the original problem, and how can it be implemented quickly?

- The first thing to do is to construct the prefix phrases and the suffix phrases of all candidates. We need to construct an almost max-flow from the suffixes to the prefixes. If a phrase is both a suffix and a prefix, then these two instances cancel out.
- First we should look for a source u and a sink x that are separated by distance 1 in the underlying graph. This is equivalent to saying that there is a length $k-2$ string that is a suffix of u and a prefix of x .
- Next we should look for a source u and a sink x that are separated by distance 2 in the underlying graph. This is equivalent to saying that there is a length $k-3$ string is a suffix of u and a prefix of x .
- And so on...

This can be summarized as follows:

- Let P = the multi-set of prefix phrases of all candidates.
- Let S = the multi-set of suffix phrases of all candidates.
- Let $x = k + |C|$ and $i = 0$.
- (*) While $|P| \geq 2$ and $|P \cap S| \geq 1$: delete one copy of a common element from both P and S and increment x by i .
- Remove the last letter from every element in P and the first letter from every element in S .
- Increment i by 1, and repeat again from (*) until P and S have just one element.
- Output x .

Unfortunately, this is still a little too slow. It will run in time proportional to the output password string length, which could be 10^{18} . The final optimization is that if you look at any element in P (or S) in the algorithm above, then all 133tspeak variations of that element should also be in P (or S). You can always treat all variations as one big batch:

- Let P = the map from phrase to the number of times a 133tspeak variation of that phrase appears as the prefix of a candidate.
- Define S similarly for suffixes.
- Let $x = k + |C|$ and $i = 0$.
- (*) While P and S are non-empty:
- While P and S have a common element t : delete $\min(P[t], S[t])$ from $P[t]$ and $S[t]$ and increment x by $i * \min(P[t], S[t])$.
- Remove the last letter from every element in P and the first letter from every element in S .
- Increment i by 1 and repeat again from (*) until P and S are empty.
- Output $x-i+1$.

The Misof Implementation

Only one contestant solved this problem during the contest, and he used a variation on the method approached here. Instead of using the greedy algorithm, he implemented the flow directly, grouping together 133t variations in the same way that we did to achieve sub-linear running time in the password string size. It is a little slower and a little harder to implement, but it works just fine. Congratulations to misof for coming up with this!

Analysis: Perfect Game

Preamble

It will come as no great surprise that this problem was inspired by a video game that the author played: Super Meat Boy. Turning the real achievement challenge into a Code Jam problem involved a little simplifying: it doesn't take the same amount of time to lose a level as to win it, for example.

Total Expected Time and Number of Attempts

Your total expected time to complete the achievement will be the expected number of attempts, times the expected time per attempt.

First, let's figure out how many attempts it's going to take us to complete the achievement. The probability of success in any given attempt is easy to compute, since it's simply the probability of succeeding on every level once without failing any of them: $(1-p_1) * (1-p_2) * \dots * (1-p_N)$.

The expected number of attempts won't depend on anything other than this probability, and this probability doesn't depend on the order of the levels; so the expected number of attempts doesn't depend on the order of the levels, and we can ignore it when figuring out what order to attempt them in.

In case you're curious about what the expected number of attempts is anyway, read on. Let's say that we're going to try to do something that might fail, like complete this achievement, and that we're going to keep trying until we succeed. If the probability of success is P , then what we're doing is called a [Bernoulli Trial](#), repeated until we achieve success. The number of attempts that will take falls into a random distribution called the [Geometric Distribution](#). The expected value of this distribution—the number of attempts it will take—is $1/P$.

Knowing about the Geometric Distribution lets us compute the expected number of attempts easily. The chance that we'll complete every level successfully, one after another, is the product of the probabilities of success, which we'll call P ; the number of attempts it will take us to complete that is $1/P$. As promised, this number doesn't depend on the order in which we attempt the levels, and since what we're trying to do is compute the best order, we're going to ignore it.

Time Per Attempt

Let's choose an arbitrary order for the levels. Suppose the i^{th} of those levels takes t_i time, and you die in that level with probability p_i . In any attempt, you'll definitely reach level 1; you'll reach level 2 with probability $1-p_1$; you'll reach level 3 with probability $(1-p_1) * (1-p_2)$; and so on.

Based on those calculations, the amount of time it takes you to make one attempt will be $\text{expected time} = t_1 + (1-p_1)t_2 + (1-p_1)(1-p_2)t_3 + \dots$. This is because you will try level i only if you pass the first $i-1$ levels.

Now, let's consider what would happen to that expected time if we swapped levels i and $i+1$. Only two of the terms of the expected time equation would be affected—the others would simply reverse the order of $(1-p_i)$ and $(1-p_{i+1})$, which doesn't matter. Those two terms themselves have several multiplicative terms in common:

Pre-swap:

$$(1-p_1)(1-p_2)\dots(1-p_{i-1})t_i + \\ (1-p_1)(1-p_2)\dots(1-p_{i-1})(1-p_i)t_{i+1}.$$

Post-swap:

$$(1-p_1)(1-p_2)\dots(1-p_{i-1})t_{i+1} + \\ (1-p_1)(1-p_2)\dots(1-p_{i-1})(1-p_{i+1})t_i.$$

So we're better off post-swap iff:

$$t_i + (1-p_i)t_{i+1} > t_{i+1} + (1-p_{i+1})t_i \\ t_i p_{i+1} > t_{i+1} p_i$$

Now we can compare two adjacent levels to see whether they should be swapped. Doing this greedily results in a [stable sort](#) of the levels. With a little more work, you can prove that non-adjacent levels should be swapped under the same conditions, and that the stable sort is thus optimal.

Implementation Details

Our last inequality above said we should swap iff $t_i p_{i+1} > t_{i+1} p_i$. It's tempting to go one step further, to $t_i/p_i > t_{i+1}/p_{i+1}$, but we can't: p_i or p_{i+1} could be zero, and dividing by zero isn't a mathematically valid thing to do.

Another reason not to make that final step—though the final step actually kind of works if you don't mind special-casing $p_i=0$ to give an infinite score—is because division almost inevitably makes us deal with floating-point numbers. In a problem like this where we're trying to make a decision based on numbers, we want to make sure we're doing exact comparisons. We wouldn't want to reverse two equivalent levels because $t_i/p_i = 10.0000000001$ and $t_{i+1}/p_{i+1} = 10.0$.

Analysis: Quality Food

Multiple Solutions

Sometimes our problemsetters like to come up with big, complicated solutions to their problems, and sometimes it turns out that there are simpler alternatives. We'll present two solutions here, including the simple one that many problems solvers used, and the more complex one used by the author.

How much does a single delivery cost?

Let's look at the cost of a delivery containing K meals. It's easy to see that we should order the cheapest meal with a time-to-stale of at least 0, the cheapest meal with a time-to-stale of at least 1, the cheapest meal with a time-to-stale of at least 2, and so on until we order the cheapest meal with a time-to-stale of at least K . Since K could be very large, we'll solve this problem in $O(N \log(N))$.

First we'll sort the kinds of food by price, and construct an array that contains $[(0, 0), (d_1, c_1), (d_2, c_2), \dots]$, where we should buy food that costs c_i for the meals between d_{i-1} and d_i after the delivery arrives. We can then process that into another array containing (d_i, C_i) , where C_i is the total cost of a single delivery that lasts d_i days, and $(C_i - C_{i-1}) / (d_i - d_{i-1}) = c_i$.

We only had to do those above steps once. Now that we have that array, it's an $O(\log(N))$ binary search (or a simpler linear search, since we'll find that we don't need the efficiency for either of our solutions) to find out how much it costs to have a delivery that lasts K days: find i such that $d_{i-1} \leq K < d_i$, and charge $C_{i-1} + (K - d_{i-1}) * c_{i-1}$.

We'll call that function `SingleDeliveryCost(days)`, and for convenience later in this editorial we'll also define `SingleDayCost(day)` to be the cost to buy the cheapest meal with a time-to-stale of at least `day`. Observe that `SingleDayCost(a) ≤ SingleDayCost(b)` if $a \leq b$, and that `SingleDeliveryCost(days+1) = SingleDeliveryCost(days) + SingleDayCost(days+1)`.

Deliveries should be (almost) the same size

Let's show that all of your deliveries should be almost the same size. Let's suppose that we have a solution that has a delivery, A, containing a meals, and another delivery, B, that contains b meals, with $b \geq a+2$. Then the cost for those two deliveries is `SingleDeliveryCost(a) + SingleDeliveryCost(b)`. If instead we increased delivery A's size to $a+1$ and decreased delivery B's size to $b-1$, we'd have the same total number of days, but the delivery cost would be:

$$\begin{aligned} & \text{SingleDeliveryCost}(a+1) + \text{SingleDeliveryCost}(b-1) \\ &= \text{SingleDeliveryCost}(a) + \text{SingleDayCost}(a+1) + \text{SingleDeliveryCost}(b) - \text{SingleDayCost}(b-1) \\ &\leq \text{SingleDeliveryCost}(a) + \text{SingleDeliveryCost}(b). \end{aligned}$$

This shows that all our deliveries should be of size a or $a+1$ for some value of a .

Deliveries should be of an "interesting" size

Let's say that we want to buy a total of D days' worth of food, and we want to know how big our deliveries should be. We'll start by considering a number of deliveries x that we're going to show is "uninteresting": one such that if we look the size of the largest delivery, $\text{ceil}(D/x)$, then $\text{ceil}(D/(x-1)) = \text{ceil}(D/x) = \text{ceil}(D/(x+1))$. In such a case, adding a delivery changes the cost by $\text{SingleDeliveryCost}(\text{ceil}(D/x)) - \text{ceil}(D/x) * \text{SingleDayCost}(\text{ceil}(D/x))$. If we add a delivery, we'll change the cost by the negation of that amount. One of those two quantities has to be non-negative, which means that we never need to consider making x deliveries.

That means that we only need to look at numbers of deliveries such that if we changed the number of deliveries, we'd be changing $\text{SingleDayCost}(\text{ceil}(D/x))$. There are only $2N$ such delivery sizes, and trying all of them solves the problem in $O(N \log(N))$.

The other approach

A competitor who couldn't convince himself or herself of the above still has a good chance of solving the problem. We'll start by presenting a trick ("Changing the Question") that you can use whether or not you have the above realization—it might make the math easier—and a totally different approach that, admittedly, requires some math of its own.

Changing the Question

An observation we can make is that instead of solving the problem that was asked, we can play a trick to let ourselves solve a simpler problem: "Can I eat quality food for D days?" We do this by **binary searching on the answer**. We know the answer is between 0 and M , so we can start by solving that problem $O(\log(M))$ times.

How many deliveries should there be?

If the logic in the previous solution was too complicated, here's an alternative that probably isn't going to make you any happier. On the other hand, if you were thinking that there weren't enough logarithmic factors in the complexity analysis, or enough -ary searches, this could make you happy. This was our original solution for the problem, and it makes the I/O author perversely proud to have been part of making a problem to which a binary search in a ternary search in a binary search is a perfectly reasonable solution.

The argument here is that we can [ternary search](#) for the number of deliveries that minimizes the cost to buy quality food for D days. In order for a ternary search to be possible, the function has to be strictly decreasing, then strictly increasing (or vice versa). In this case, the domain of the function is from where the number of deliveries x is the minimum possible number at which we could order D meals (regardless of how expensive they are), to $x=D$. We'll show that the function decreases, then increases over this domain.

First let's take the `SingleDayCost` function and extend it over the real numbers. We already know what it is for the integers; let's just linearly interpolate for the real numbers in between, and call the result G . This has the nice result that the cost for x deliveries and D days, which we'll call $H(x)$, is $H(x) = G(D/x) * x + x * F$.

Now, we're about to start taking derivatives, and although G is continuous, it isn't differentiable. There are a couple of ways of getting around this, and we're going to take a physicists' route. We'll define a function G'' to be a sum of [delta functions](#) such that the double-integral of G'' over x is our original G . Then we'll define G' to be G'' 's integral, and G to be G' 's integral. That lets us say that the first derivative of G is non-negative everywhere, and so is its second derivative. Don't try this at home, folks, especially if you live in a math class.

What we're really interested in doing here is proving that $H(X)$ is decreasing and then increasing, which we can do by proving that it has a positive second derivative:

$$H(X) = G(D/X) * X + X * F$$

$$H'(X) = G(D/X) - X * G'(D/X) * X^{-2} + F$$

$$H'(X) = G(D/X) - G'(D/X) / X + F$$

$$H''(X) = -X^{-2} G'(D/X) + X^{-2} G'(D/X) + G''(D/X) X^{-3}$$

$$H''(X) = G''(D/X) X^{-3} \geq 0$$

Therefore $H(X)$ is decreasing and then increasing (or just decreasing or increasing, which is also fine), and can be ternary searched on for the minimum cost of surviving **D** days. This algorithm's complexity works out to $O(N \log(N) + \log^2(M) \log(N))$.